

Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

All the trip files was uploaded in the google colab eda folder
an output path was defined and cleared any preexisting file
loop was run with each file as iteration
The record count was taken for each file.
dataframe was made from the file as per iteration
pickup_date and pickup_hour columns was added in the dataframe
sampled the data from datafram grouped by pickup_date and
pickup_hour
sampled dataframe was then appended in a csv file in output path
defined. The same loop was run for all all trip files.

```
#print(f"Number of rows in '2023-1.parquet': {len(df_uploaded_file)}")

#to take a look of number of records in each of the 12 files provided

input_path="/content/eda/2023*.parquet"
parquet_files = glob.glob(input_path)

for file_path in parquet_files:
    df = pd.read_parquet(file_path, engine="pyarrow")
    print(f"number of records in the file {file_path}: {len(df)}")
```

```
number of records in the file /content/eda/2023-5.parquet: 2889185
number of records in the file /content/eda/2023-3.parquet: 3275796
number of records in the file /content/eda/2023-6.parquet: 3258261
number of records in the file /content/eda/2023-11.parquet: 3302857
number of records in the file /content/eda/2023-1.parquet: 3041714
number of records in the file /content/eda/2023-12.parquet: 3333925
number of records in the file /content/eda/2023-8.parquet: 2875947
number of records in the file /content/eda/2023-7.parquet: 3481547
number of records in the file /content/eda/2023-10.parquet: 3485185
number of records in the file /content/eda/2023-4.parquet: 2792901
number of records in the file /content/eda/2023-9.parquet: 2817156
number of records in the file /content/eda/2023-2.parquet: 3374086
```

```

# Take a small percentage of entries from each hour of every date.
# Iterating through the monthly data:
# read a month file -> day -> hour: append sampled data -> move to next hour -> move to next day after 24 hours -> move to next month file
# Create a single dataframe for the year combining all the monthly data

input_path="/content/eda/2023*.parquet"
output_file="/content/sample_data/eda_output/sample_data.csv"

parquet_file=glob.glob(input_path)

if os.path.exists(output_file):
    os.remove(output_file)

writer= None

for i, parquet_file in enumerate(parquet_file):

    print("Processing the file : ", {parquet_file})

    df = pd.read_parquet(parquet_file, engine="pyarrow")

    df["pickup_date"] = df["tpep_pickup_datetime"].dt.date
    df["pickup_hour"] = df["tpep_pickup_datetime"].dt.hour

    sample_dataframe=(df.groupby(["pickup_date", "pickup_hour"], group_keys=False).apply(lambda x: x.sample(frac=0.007, random_state=42)))

    #sample_5df=df.sample(frac=.05, random_state=42)

    sample_dataframe.to_csv(
        output_file,
        mode="a",
        header=(i == 0),
        index=False
    )

print("final file with sample data from each file is created")

processing the file : {'/content/eda/2023-5.parquet'}
tmp/ipython-input-673786775.py:25: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a
o exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
    sample_dataframe=(df.groupby(["pickup_date", "pickup_hour"], group_keys=False).apply(lambda x: x.sample(frac=0.007, random_state=42)))
processing the file : {'/content/eda/2023-3.parquet'}
tmp/ipython-input-673786775.py:25: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a
o exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
    sample_dataframe=(df.groupby(["pickup_date", "pickup_hour"], group_keys=False).apply(lambda x: x.sample(frac=0.007, random_state=42)))
processing the file : {'/content/eda/2023-6.parquet'}
tmp/ipython-input-673786775.py:25: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a
o exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
    sample_dataframe=(df.groupby(["pickup_date", "pickup_hour"], group_keys=False).apply(lambda x: x.sample(frac=0.007, random_state=42)))
processing the file : {'/content/eda/2023-11.parquet'}

```

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

the data was clean initially hence no action was taken to to perform data cleaning. At later point of time, if required, the data cleansing would be done.

2.1.2. Combine the two airport_fee columns

Based on the method used to read each file one at a time and sample the data .007 fraction at a time then writing the same into the csv file. This way the first file header is used in the csv file. we see that 2023-1.parquet file has airtpot_fee in small and all other files have Airport_fee. Hence the csv contains only Airport_fee column only. so no action is required to merge the columns and we see only 1 Airport_fee column created in the sampled data.

```

14] df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 265487 entries, 0 to 265486
Data columns (total 21 columns):
 #   Column              Non-Null Count  Dtype  
---  -
 0   VendorID            265487 non-null  int64  
 1   tpep_pickup_datetime 265487 non-null  object  
 2   tpep_dropoff_datetime 265487 non-null  object  
 3   passenger_count      256656 non-null  float64  
 4   trip_distance        265487 non-null  float64  
 5   RatecodeID          256656 non-null  float64  
 6   store_and_fwd_flag   256656 non-null  object  
 7   PULocationID         265487 non-null  int64  
 8   DOLocationID         265487 non-null  int64  
 9   payment_type         265487 non-null  int64  
10   fare_amount          265487 non-null  float64  
11   extra                265487 non-null  float64  
12   mta_tax              265487 non-null  float64  
13   tip_amount           265487 non-null  float64  
14   tolls_amount         265487 non-null  float64  
15   improvement_surcharge 265487 non-null  float64  
16   total_amount         265487 non-null  float64  
17   congestion_surcharge 256656 non-null  float64  
18   Airport_fee          256656 non-null  float64  
19   pickup_date          265487 non-null  object  
20   pickup_hour          265487 non-null  int64  
dtypes: float64(12), int64(5), object(4)
memory usage: 42.5+ MB

```

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

IsNull function in dataframe was used to identify the missing values in each column of the dataframe. There was columnwise null values counted and then its percentage was derived on the total number of records in the column.

```

      column  negative_values
0    fare_amount              0
1      extra                  1
2    mta_tax                  11
3    tip_amount              0
4    tolls_amount            0
5  improvement_surcharge      11
6    total_amount            11
7  congestion_surcharge       6
8    Airport_fee              2

```

2.2.2. Handling missing values in passenger_count

Missing value in the passenger_count was replaced with MODE value of the passenger_count. The passenger_count is categorical variable hence the missing value is replaced with MODE value to maintain datasize and prevent algorithm errors.

passenger_count value 0 was replaced with 1 as minimum 1 passenger would be present in any ride.

	Missing Values In Column	Missing Values Percentage
passenger_count	8831	3.32634
RatecodeID	8831	3.32634
store_and_fwd_flag	8831	3.32634
congestion_surcharge	8831	3.32634
Airport_fee	8831	3.32634

Record with null value in column 'passenger_count':

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	\
12	2 2023-02-01 01:46:13	2023-02-01 01:59:11	NaN	
21	1 2023-02-01 05:28:00	2023-02-01 05:33:34	NaN	
35	2 2023-02-01 06:57:26	2023-02-01 07:04:46	NaN	
41	2 2023-02-01 07:57:51	2023-02-01 08:23:13	NaN	
58	2 2023-02-01 07:51:43	2023-02-01 08:03:48	NaN	

	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	\
12	2.98	NaN	NaN	114	33	
21	1.40	NaN	NaN	141	162	
35	3.87	NaN	NaN	75	179	
41	4.05	NaN	NaN	239	234	
58	2.13	NaN	NaN	143	246	

	payment_type	...	extra	mta_tax	tip_amount	tolls_amount	\
12	0	...	0.0	0.5	5.15	0.00	
21	0	...	1.0	0.5	2.72	0.00	
35	0	...	0.0	0.5	4.28	6.55	
41	0	...	0.0	0.5	3.94	0.00	
58	0	...	0.0	0.5	3.94	0.00	

	improvement_surcharge	total_amount	congestion_surcharge	Airport_fee	\
12	1.0	30.92	NaN	NaN	
21	1.0	16.32	NaN	NaN	
...					

Number of records with passenger count mentioned as 0: 4070
Number of records with passenger count mentioned as 0 updated to 1: 0
count of null 'passenger_count' after replacement of null values and 0 value in passenger count with MODE and 1: 0

2.2.3. Handle missing values in RatecodeID

replacing the missing value in RateCodeId by MODE as it defines the mathematical support to replace the NULL values in the column.

```

Distribution of 'RatecodeID':
RatecodeID
1.0      242253
2.0      10081
NaN       8831
99.0      1530
5.0       1428
3.0        845
4.0        519
Name: count, dtype: int64
missing or null values in RateCodeId is replaced with mode: 1.0

Null values in 'RatecodeID' after it is replaced with MODE value: 0

```

2.2.4. Impute NaN in congestion_surcharge

with respect to surcharge Nan may infer to no surcharge. hence it can be replaced with 0

```

pattern distribution of 'congestion_surcharge':
congestion_surcharge
2.5      236938
0.0      19712
NaN       8831
-2.5         6
Name: count, dtype: int64

missing values in 'congestion_surcharge' is replaced with 0.0
missing values in 'congestion_surcharge' after replacement with 0: 0

```

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

count, mean, standard deviation, minimum, maximum and quartiles were calculated on the sample dataframe.

```

count      mean      std      min      25%      50%      \
VendorID      265487.0      1.733475      0.447562      1.00      1.00      2.00
passenger_count      265487.0      1.370538      0.862992      1.00      1.00      1.00
trip_distance      265487.0      3.581805      48.595892      0.00      1.05      1.80
RatecodeID      265487.0      1.636491      7.422751      1.00      1.00      1.00
PULocationID      265487.0      165.172668      64.070411      1.00      132.00      162.00
DOLocationID      265487.0      163.952988      69.779730      1.00      113.00      162.00
payment_type      265487.0      1.164980      0.506647      0.00      1.00      1.00
fare_amount      265487.0      19.830154      18.204986      0.00      9.30      13.50
extra      265487.0      1.587679      1.824932      -2.50      0.00      1.00
mta_tax      265487.0      0.495325      0.048416      -0.50      0.50      0.50
tip_amount      265487.0      3.548453      4.046095      0.00      1.00      2.84
tolls_amount      265487.0      0.593724      2.178536      0.00      0.00      0.00
improvement_surcharge      265487.0      0.999003      0.030623      -1.00      1.00      1.00
total_amount      265487.0      28.891858      22.769544      -4.00      15.96      21.02
congestion_surcharge      265487.0      2.231107      0.774734      -2.50      2.50      2.50
Airport_fee      265487.0      0.138263      0.457699      -1.75      0.00      0.00
pickup_hour      265487.0      14.265844      5.806733      0.00      11.00      15.00

75%      max
VendorID      2.00      6.00
passenger_count      1.00      9.00
trip_distance      3.40      22528.82
RatecodeID      1.00      99.00
PULocationID      234.00      265.00
...
look for trips with 0 distance and non zero fare
5226
checking for distnce of trip < 1 km and trip fare >200
5

```

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

Classify variables into categorical and numerical
Categorise the variables into Numerical or Categorical.

Categorise the variables into Numerical or Categorical.

VendorID: Categorical

tpep_pickup_datetime: Numerical

tpep_dropoff_datetime: Numerical

passenger_count: Numerical

trip_distance: Numerical

RatecodeID: Categorical

PULocationID: Categorical

DOLocationID: Categorical

payment_type: Categorical

pickup_hour: Categorical

trip_duration: Numerical

The following monetary parameters belong in the same category, is it categorical or numerical?

fare_amount - Numerical

extra - Numerical

mta_tax - Numerical

tip_amount - Numerical

tolls_amount - Numerical

improvement_surcharge - Numerical

total_amount - Numerical

congestion_surcharge - Numerical

airport_fee - Numerical

3.1.1. Analyse the distribution of taxi pickups by hours, days of the week, and months

The sampled data was analyzed for hourly pickup, trips distribution over the days of the week and month.

The dataframe was analyzed to identify the number of 0 in fare_amount, tip_amount, total-amount, trip-distance.

3.1.1.1. Filter out the zero/negative values in fares, distance and tips

Data with 0 as fare_amount, 0 as tip_amount and 0 as trip distance was removed from the dataframe.

```
# Create a df with non zero entries for the selected parameters.
# Create a copy of the dataframe which was created over the sampled data
df_financial = df.copy()

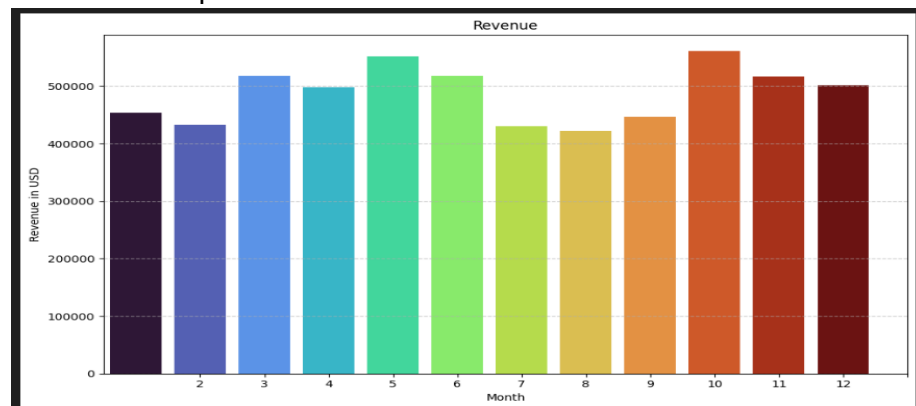
# filtering the records where fare_amount, tip_amount, total_amount, or trip_distance are zero
# Considering a 0 tip is a valid outcome, but for 'analysis of tipping behaviour' it can be useful to filter.
df_financial = df_financial[df_financial['total_amount'] > 0]
df_financial = df_financial[df_financial['fare_amount'] > 0]
df_financial = df_financial[df_financial['tip_amount'] > 0]
df_financial = df_financial[df_financial['trip_distance'] > 0]

print(f"number of records in original dataframe: {len(df)}")
print(f"number of records in financial dataframe after removing the 0 value records: {len(df_financial)}")
```

```
number of records in original dataframe: 252130
number of records in financial dataframe after removing the 0 value records: 198056
```

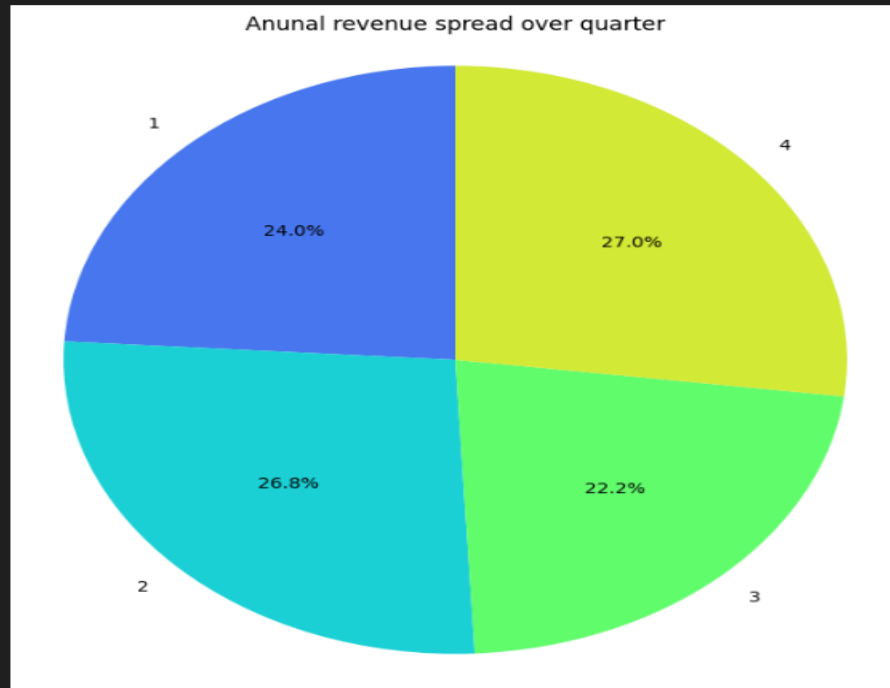
3.1.2. Analyse the monthly revenue trends

Bar chart was plotted to see the month wise revenue distribution.

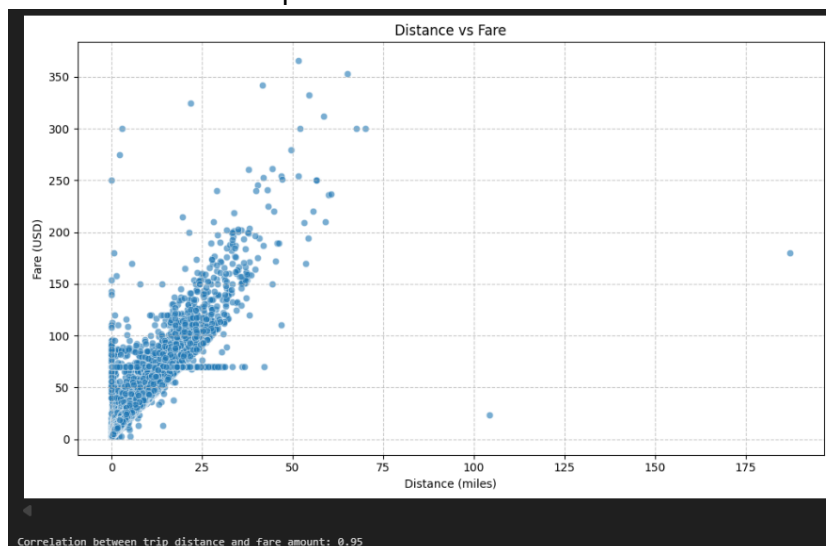


- 3.1.3. Find the proportion of each quarter's revenue in the yearly revenue
- Annual revenue spread over the quarter was plotted in a pie chart to see the % share in total revenue.

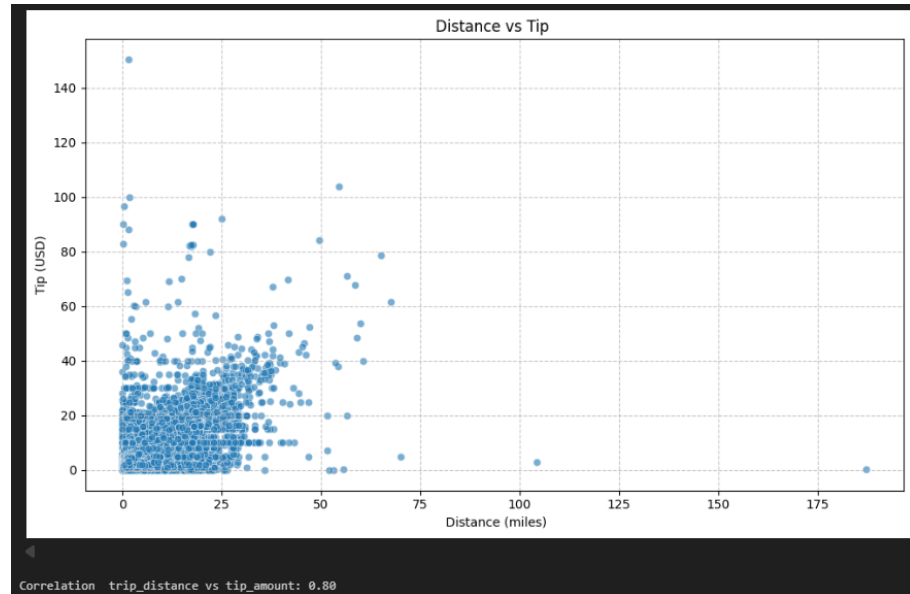
```
Revenue Proportion per quarter(%):
quarter
1    1404810.04
2    1568571.55
3    1300214.52
4    1579849.10
Name: total_amount, dtype: float64
```



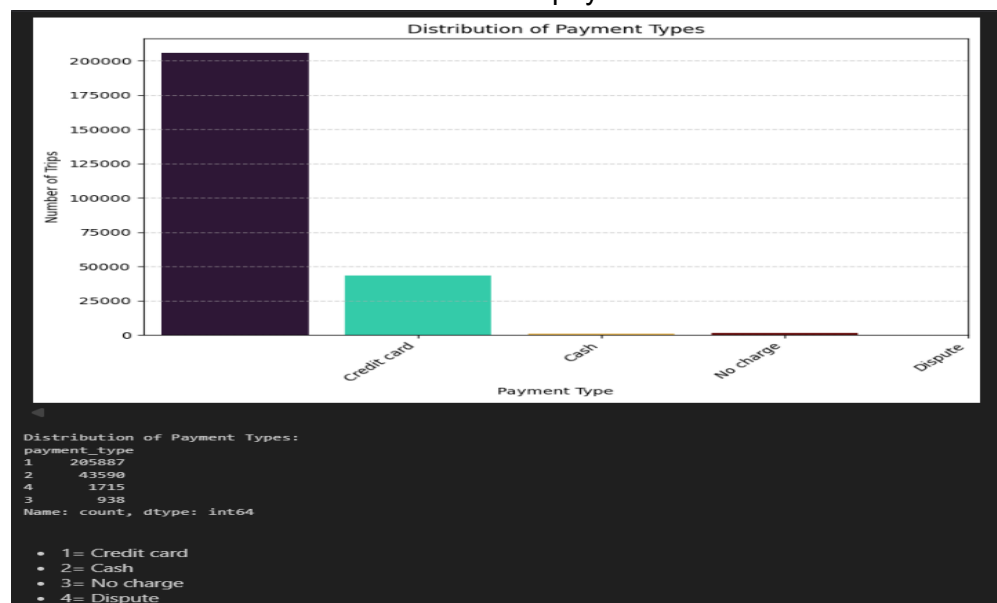
- 3.1.4. Analyse and visualise the relationship between distance and fare amount
- distance vs fare was plotted and correlation was established to be .95



- 3.1.5. Analyse the relationship between fare/tips and trips/passengers**
distance was plotted with tip value and a .8 correlation was found to exist

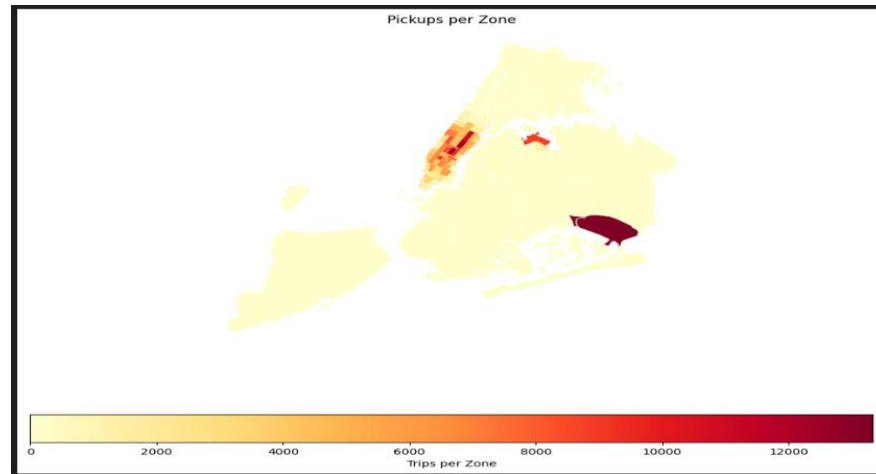


- 3.1.6. Analyse the distribution of different payment types**
revenue with payment type was plotted in bar graph and an observation was made that most payments were made from credit card. Credit card came out to be most favorable method of payment.



3.1.7. Load the taxi zones shapefile and display it

pickup zone wise data was plotted to see the most favorable zone of pickup.



3.1.8. Merge the zone data with trips data

the trips data was merged with the zone data based on the locationId and PULocationID.

```
# Merge zones and trip records using locationID and PULocationID

# Merging the zones and trip records by using locationID and PULocationID in zones and sample dataframe
merged_df = pd.merge(df, zones, left_on='PULocationID', right_on='LocationID', how='left')
merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 252137 entries, 0 to 252136
Data columns (total 31 columns):
#   Column                Non-Null Count  Dtype
---  -
0   VendorID              252137 non-null  int64
1   tpep_pickup_datetime  252137 non-null  datetime64[ns]
2   tpep_dropoff_datetime 252137 non-null  datetime64[ns]
3   passenger_count       252137 non-null  float64
4   trip_distance         252137 non-null  float64
5   RatecodeID            252137 non-null  float64
6   store_and_fwd_flag    252137 non-null  object
7   PULocationID          252137 non-null  int64
8   DOLocationID          252137 non-null  int64
9   payment_type          252137 non-null  int64
10  fare_amount           252137 non-null  float64
11  extra                 252137 non-null  float64
12  mta_tax               252137 non-null  float64
13  tip_amount            252137 non-null  float64
14  tolls_amount          252137 non-null  float64
15  improvement_surcharge 252137 non-null  float64
16  total_amount          252137 non-null  float64
17  congestion_surcharge  252137 non-null  float64
18  Airport_fee           252137 non-null  float64
19  pickup_date           252137 non-null  object
...
29  borough               249898 non-null  object
30  geometry               249898 non-null  geometry
dtypes: datetime64[ns](2), float64(17), geometry(1), int32(1), int64(5), object(5)
memory usage: 58.7+ MB
```

3.1.9. Find the number of trips for each zone/location ID

the number of trips with location id has been charted in descending order to see top 10 location id.

```
Top 10 zones by trip count:
PULocationID  trip_count
0            132      13336
1            237      12065
2            161      11954
3            236      10749
4            162       9230
5            138       8901
6            186       8679
7            230       8486
8            142       8306
9            170       7493
```

3.1.10. Add the number of trips for each zone to the zones dataframe

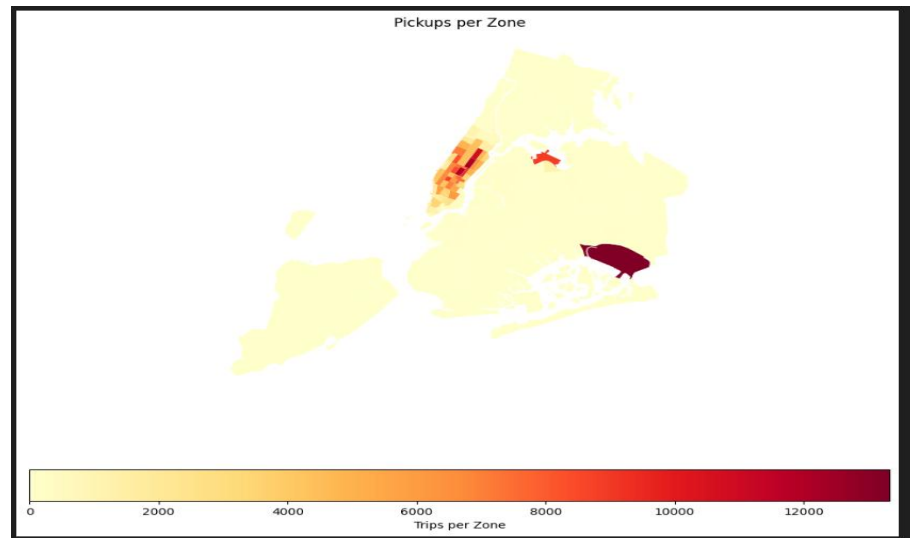
```
Zones GeoDataFrame with trip counts:
OBJECTID  Shape_Leng  Shape_Area  zone  LocationID  \
0         1    0.116357    0.000782  Newark Airport      1
1         2    0.433470    0.004866  Jamaica Bay      2
2         3    0.084341    0.000314  Allerton/Pelham Gardens  3
3         4    0.043567    0.000112  Alphabet City     4
4         5    0.092146    0.000498  Arden Heights     5

borough  geometry  \
0  EWR POLYGON ((933100.918 192536.086, 933091.011 19...
1  Queens MULTIPOLYGON (((1033269.244 172126.008, 103343...
2  Bronx POLYGON ((1026308.77 256767.698, 1026495.593 2...
3  Manhattan POLYGON ((992073.467 203714.076, 992068.667 20...
4  Staten Island POLYGON ((935843.31 144283.336, 936046.565 144...

PULocationID  trip_count
0             1.0         5.0
1            NaN         0.0
2            NaN         0.0
3             4.0        238.0
4            NaN         0.0
```

3.1.11. Plot a map of the zones showing number of trips

the heat map was plotted for the above ask



Conclude with results

*** Busiest hours, days and months**

Busy Hour – 2-7 PM

Busy Day – Fri/Sat/Sun forms the busiest days with trips

Busy Month – pickup count wise Mar/Apr/May/Jun/Oct/Nov/Dec forms the busiest month

*** Trends in revenue collected**

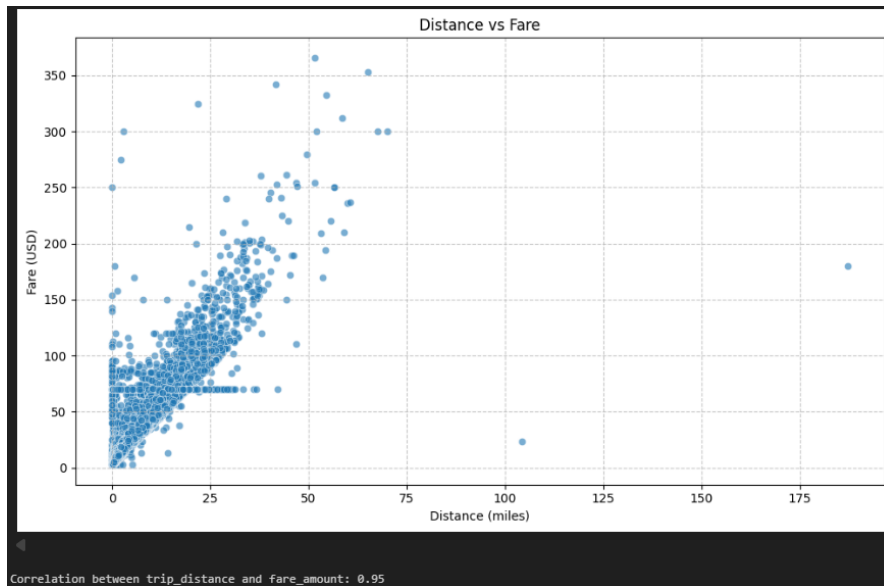
Revenue wise Mar/May/June/Oct/Nov gives high revenue. Plot attached above for reference and conclusion

*** Trends in quarterly revenue**

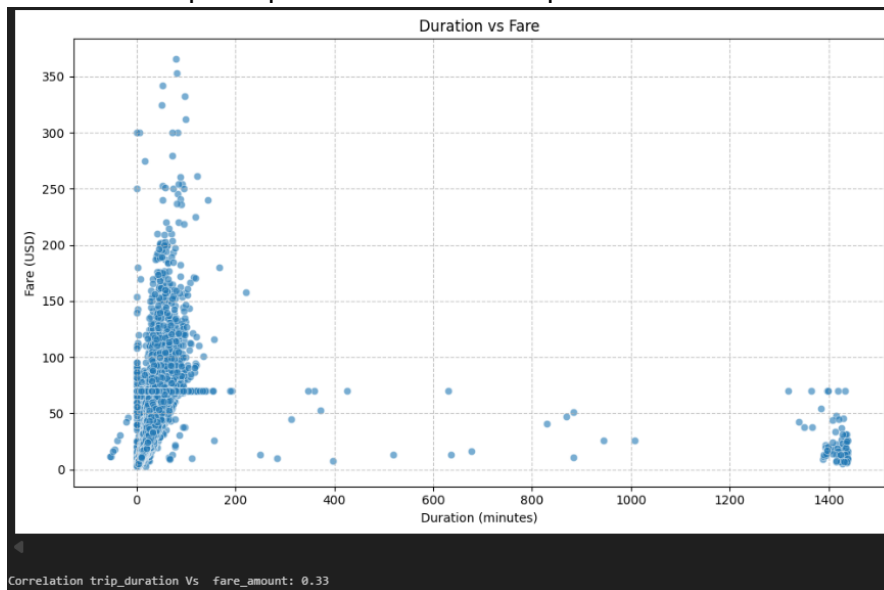
Q2 and Q4 contribute high towards revenue

*** How fare depends on trip distance, trip duration and passenger counts**

Based on the correlation chart of the distance vs fare we see that it is positively correlated.

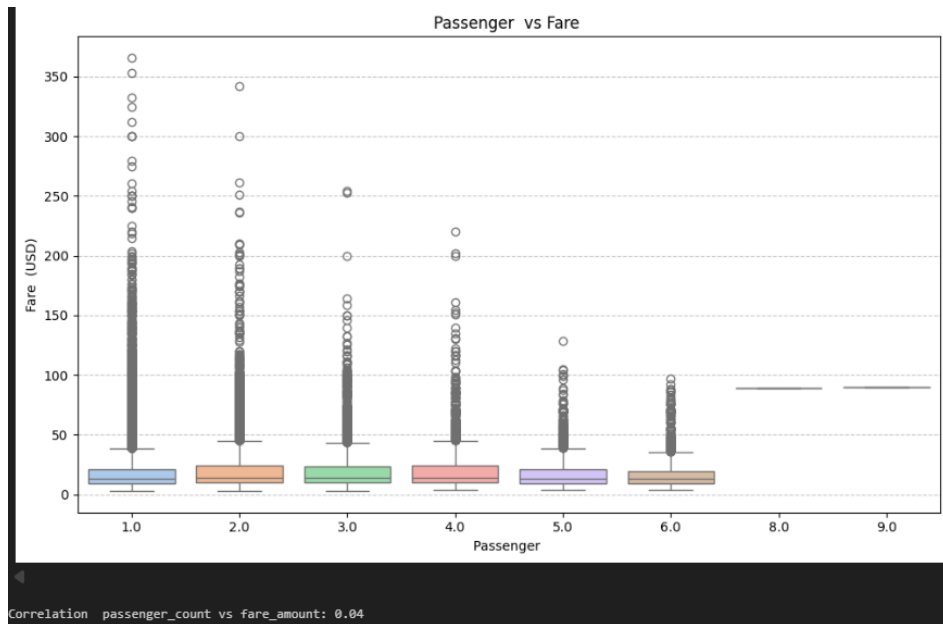


Based on the plot trip duration and fare a positive correlation was observed.



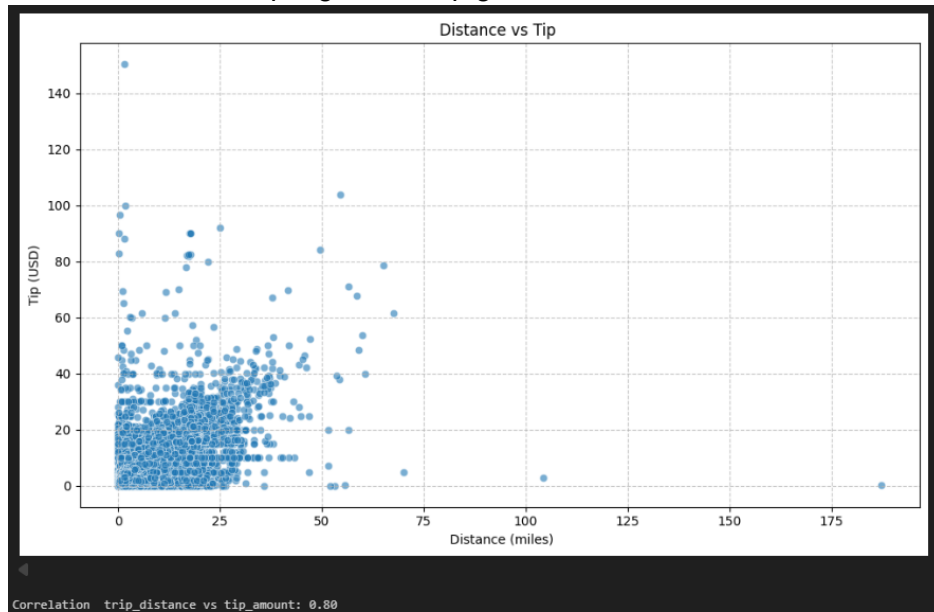
Based on the correlation between passenger count and fare amount, a very

small correlation was observed.



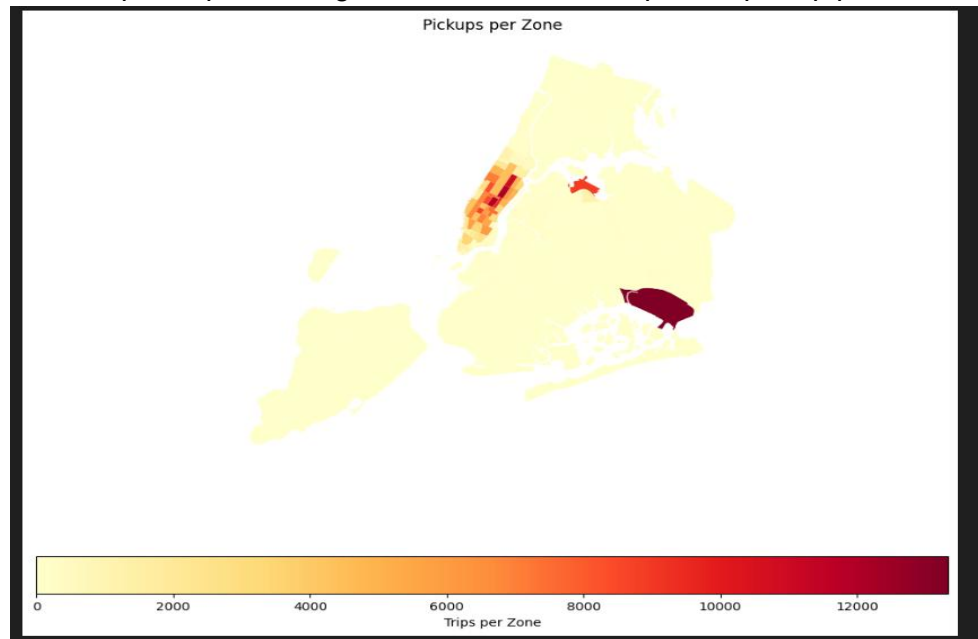
* How tip amount depends on trip distance

Plotting the trip distance with tip value we see a strong correlation between them. Short is the trip high is the tip given.



* Busiest zones

Heat map was plotted to get the busiest zone as per the pickup points.



3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

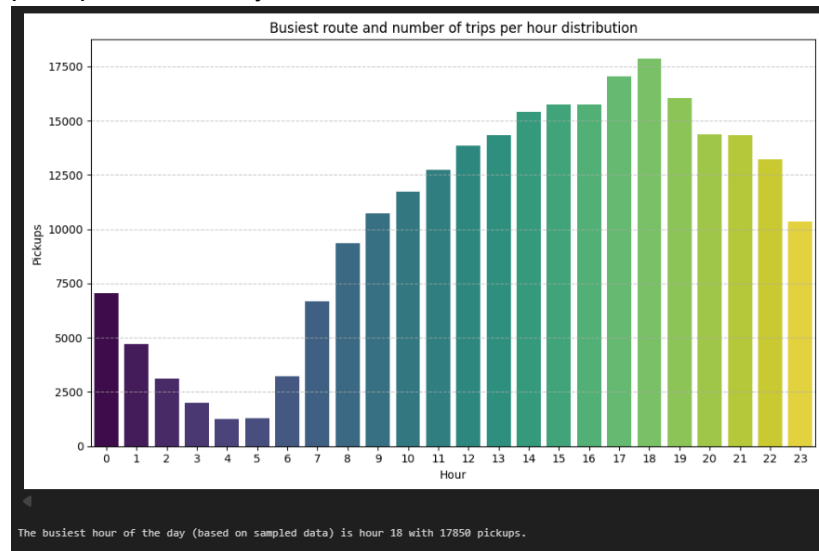
route with the average speed was arranged in descending order to identify the slow moving routes.

```
Top Slowest Routes (by average speed):
```

	Dropoff Zone	pickup_hour	avg_speed	trip_count
0	West Village	1	7.67	6
1	Clinton East	16	9.93	8
2	East Chelsea	17	12.90	8
3	East Chelsea	18	12.47	8
4	East Village	18	18.27	8
...	...	...	...	...
12617	NaN	19	10.78	107
12618	NaN	20	13.81	102
12619	NaN	21	20.47	118
12620	NaN	22	13.71	101
12621	NaN	23	14.42	71

[12622 rows x 4 columns]

- 3.2.2. Calculate the hourly number of trips and identify the busy hours** based on the sampled data a bar graph was plotted to see the number of pickups and hour of the day. We find that 2-7 PM gets the maximum pickups in each day.

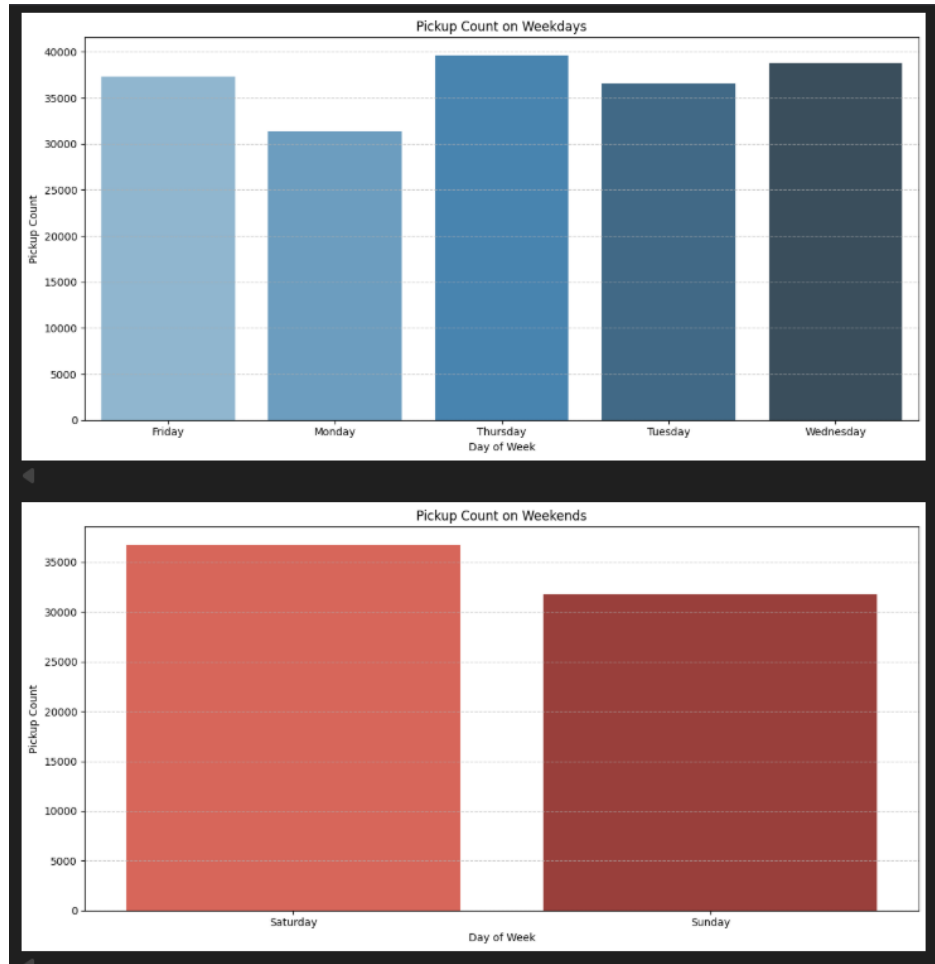


- 3.2.3. Scale up the number of trips from above to find the actual number of trips**
Count of number of pickups was plotted against the hour of the day to identify the busiest hours.

count	
pickup_hour	
23	10369
22	13207
21	14318
20	14380
19	16048
18	17850
17	17052
16	15731
15	15749
14	15394
13	14342
12	13842
11	12747
10	11722
9	10713
8	9338
7	6686
6	3232
5	1276
4	1266
3	2003
2	3093
1	4720
0	7052

3.2.4. Compare hourly traffic on weekdays and weekends

based on the datavisualization, we see that the weekdays are busy compared to weekend. People sit back and relax ver the weekends and hence less pickup counts on weekday.



3.2.5. Identify the top 10 zones with high hourly pickups and drops

below table shows the relation between hourly pickup and drops

Top 10 Pickup Zones by Trip Count:		
	PULocationID	trip_count
0	132	13336
1	237	12065
2	161	11954
3	236	10749
4	162	9230
5	138	8901
6	186	8679
7	230	8486
8	142	8306
9	170	7493

Top 10 Dropoff Zones by Trip Count:		
	DOLocationID	trip_count
0	236	11420
1	237	10700
2	161	9953
3	230	7729
4	170	7566
5	162	7254
6	142	7224
7	239	7128
8	141	6610
9	68	6500

3.2.6. Find the ratio of pickups and dropoffs in each zone

Top 10 Pickup/Dropoff Ratios (High Pickup, Low Dropoff):	
zone	trip_count_pickup trip_count_dropoff \
69 East Elmhurst	1164.0 128.0
131 JFK Airport	13336.0 2661.0
137 LaGuardia Airport	8901.0 3020.0
185 Penn Station/Madison Sq West	8679.0 5725.0
42 Central Park	4372.0 3127.0
248 West Village	5701.0 4166.0
113 Greenwich Village South	3324.0 2468.0
161 Midtown East	9230.0 7254.0
160 Midtown Center	11954.0 9953.0
99 Garment District	4145.0 3481.0
pickup_dropoff_ratio	
69	9.093750
131	5.011650
137	2.947351
185	1.515983
42	1.398145
248	1.368459
113	1.346840
161	1.272401
160	1.201045
99	1.190750
...	
197	0.016260
60	0.017045
123	0.018182
176	0.018519

3.2.7. Identify the top zones with high traffic during night hours

Number of trips during night hours (11 PM - 5 AM): 26779																				
VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_ford_flag	PULocationID	DOLocationID	payment_type	--	month	trip_duration	OBTCTD	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry	trip_id
0	Z	2023-02-01 00:14:06	2023-02-01 00:19:52	3.0	1.25	1.0	N	234	114	1	--	2	5.766667	234.0	0.036072	0.000073	Union Sq	234.0	Manhattan	DOJCUON 0863061847 20102.309, 98748.27 206.
1	T	2023-02-01 00:54:09	2023-02-01 01:13:13	1.0	5.00	1.0	N	48	87	1	--	2	19.066667	48.0	0.034747	0.000094	Clinton East	48.0	Manhattan	DOJCUON 21463.586, 98556.184 21.
2	Z	2023-02-01 00:14:36	2023-02-01 00:59:18	1.0	24.04	1.0	N	114	121	1	--	2	44.666667	114.0	0.031727	0.000047	Greeenwich Village South	114.0	Manhattan	DOJCUON 086306712 20112.788, 98330.242 20.
3	T	2023-02-01 00:49:25	2023-02-01 00:53:12	1.0	0.60	1.0	N	246	68	1	--	2	3.783333	246.0	0.069467	0.000281	West Chelsea/Hudson Yards	246.0	Manhattan	DOJCUON 080301177 21718.306, 98349.312 216.
4	T	2023-02-01 00:16:02	2023-02-01 00:22:02	1.0	0.90	1.0	N	114	148	1	--	2	6.000000	114.0	0.031727	0.000047	Greeenwich Village South	114.0	Manhattan	DOJCUON 086306712 20112.788, 98330.242 20.
3 rows x 20 columns																				

3.2.8. Find the revenue share for nighttime and daytime hours

Number of trips during night hours (11 PM - 5 AM): 29779

VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_ford_flag	PULocationID	DOLocationID	payment_type	...	month	trip_duration	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry	tr
0	2	2023-02-01 00:14:06	2023-02-01 00:19:52	3.0	1.25	1.0	N	234	114	1	...	2	5.766667	234.0	0.036072	0.000073	Union Sq	234.0	Manhattan	POLYGON ((987038.847 267023.296, 987048.27 306...
1	1	2023-02-01 00:54:09	2023-02-01 01:13:13	1.0	5.00	1.0	N	46	87	1	...	2	19.066667	46.0	0.043747	0.000094	Clinton East	46.0	Manhattan	POLYGON ((986694.313 214463.846, 986556.154 21...
2	2	2023-02-01 00:14:36	2023-02-01 00:59:16	1.0	24.04	1.0	N	114	121	1	...	2	44.666667	114.0	0.031727	0.000047	Greenwich Village South	114.0	Manhattan	POLYGON ((986306.712 203122.786, 986300.242 20...
3	1	2023-02-01 00:49:25	2023-02-01 00:53:12	1.0	0.60	1.0	N	246	68	1	...	2	3.783333	246.0	0.069467	0.000031	West Chelsea/Hudson Yards	246.0	Manhattan	POLYGON ((983021.177 217138.506, 983640.32 216...
4	1	2023-02-01 00:16:02	2023-02-01 00:22:02	1.0	0.90	1.0	N	114	148	1	...	2	6.000000	114.0	0.031727	0.000047	Greenwich Village South	114.0	Manhattan	POLYGON ((986306.712 203122.786, 986300.242 20...

5 rows x 32 columns

3.2.9. For the different passenger counts, find the average fare per mile per passenger

Shape of df_fare_analysis after filtering: (19896, 24)

VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_ford_flag	PULocationID	DOLocationID	payment_type	...	total_amount	improvement_surcharge	total_amount	congestion_surcharge	Airport_fee	pickup_date	pickup_hour	day	
0	2	2023-02-01 00:14:06	2023-02-01 00:19:52	3.0	1.25	1.0	N	234	114	1	...	0.0	1.0	18.72	2.5	0.0	2023-02-01	0	We
2	2	2023-02-01 00:14:36	2023-02-01 00:59:16	1.0	24.04	1.0	N	114	121	1	...	0.0	1.0	102.20	2.5	0.0	2023-02-01	0	We
3	1	2023-02-01 00:49:25	2023-02-01 00:53:12	1.0	0.60	1.0	N	246	68	1	...	0.0	1.0	11.80	2.5	0.0	2023-02-01	0	We
4	1	2023-02-01 00:16:02	2023-02-01 00:22:02	1.0	0.90	1.0	N	114	148	1	...	0.0	1.0	15.45	2.5	0.0	2023-02-01	0	We
5	2	2023-02-01 00:08:39	2023-02-01 00:10:30	3.0	0.53	1.0	N	263	263	1	...	0.0	1.0	11.18	2.5	0.0	2023-02-01	0	We

5 rows x 24 columns

3.2.10. Find the average fare per mile by hours of the day and by days of the week

VendorID	trip_pickup_datetime	trip_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_ford_flag	PULocationID	DOLocationID	payment_type	improvement_surcharge	total_amount	congestion_surcharge	Airport Fee	pickup_date	pickup_hour	dayofweek	month	
0	2	2023-02-01 00:14:06	2023-02-01 00:19:52	3.0	1.25	1.0	N	234	114	1	1.0	18.72	2.5	0.0	2023-02-01	0	Wednesday	month
2	2	2023-02-01 00:14:36	2023-02-01 00:59:16	1.0	24.04	1.0	N	114	121	1	1.0	102.20	2.5	0.0	2023-02-01	0	Wednesday	
3	1	2023-02-01 00:49:25	2023-02-01 00:53:12	1.0	0.60	1.0	N	246	68	1	1.0	11.80	2.5	0.0	2023-02-01	0	Wednesday	
4	1	2023-02-01 00:16:02	2023-02-01 00:22:02	1.0	0.90	1.0	N	114	148	1	1.0	15.45	2.5	0.0	2023-02-01	0	Wednesday	
5	2	2023-02-01 00:08:39	2023-02-01 00:10:30	3.0	0.53	1.0	N	263	263	1	1.0	11.18	2.5	0.0	2023-02-01	0	Wednesday	

5 rows x 25 columns

3.2.11. Analyse the average fare per mile for the different vendors

```

** Average Fare Per Mile by Vendor:
   VendorID  fare_per_mile
0           1      7.837783
1           2     9.938359

```

3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

```

Distribution of distance tiers:
distance_tier
Medium Trip (1-3 mi)      99805
Short Trip (0-1 mi)       42510
Long Trip (3-10 mi)       39574
Very Long Trip (>10 mi)   16167
Name: count, dtype: int64

First 5 rows with new 'distance_tier' column:
  trip_distance  distance_tier
0          1.25  Medium Trip (1-3 mi)
2          24.04  Very Long Trip (>10 mi)
3           0.60   Short Trip (0-1 mi)
4           0.90   Short Trip (0-1 mi)
5           0.53   Short Trip (0-1 mi)

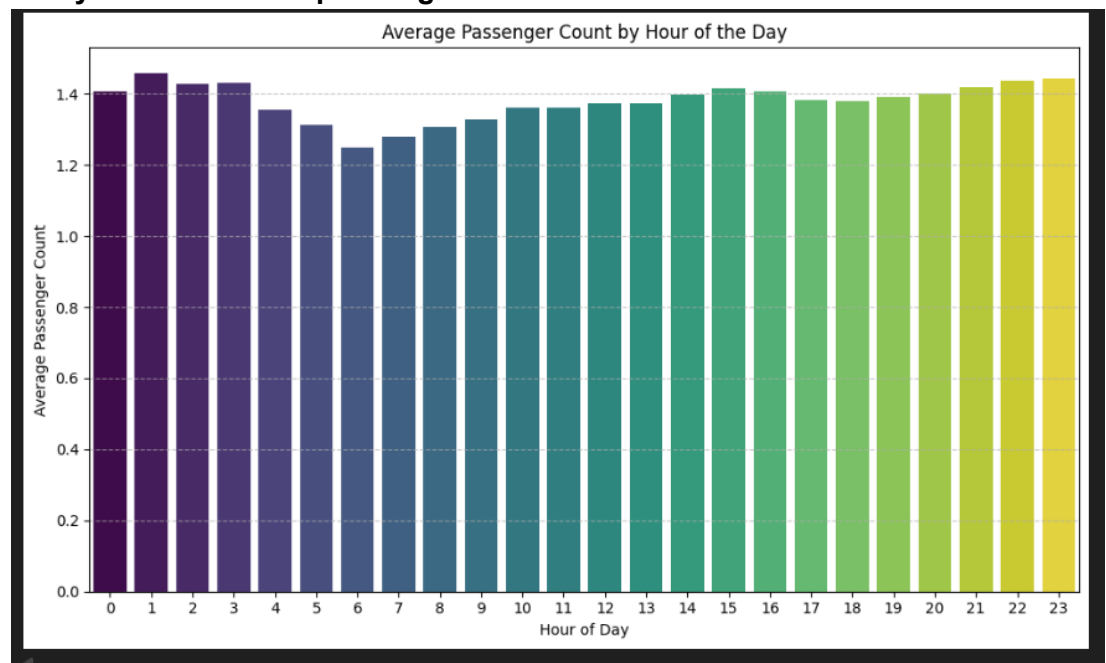
```

3.2.13. Analyse the tip percentages

VendorID	type	pickup_datetime	dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	...	congestion_surcharge	Airport_fee	pickup_date	pickup_hour	dayofweek	month	quarter	fare_per_mile	distance_tier
0	2	2023-02-01 00:14:06	2023-02-01 00:19:52	3.0	1.25	1.0	N	234	114	1	..	2.5	0.0	2023-02-01	0	Wednesday	2	1	6.880000	Medium Trip (1-3 mi)
2	2	2023-02-01 00:14:36	2023-02-01 00:59:16	1.0	24.04	1.0	N	114	121	1	..	2.5	0.0	2023-02-01	0	Wednesday	2	1	3.793677	Very Long Trip (>10 mi)
3	1	2023-02-01 00:49:25	2023-02-01 00:53:12	1.0	0.60	1.0	N	240	68	1	..	2.5	0.0	2023-02-01	0	Wednesday	2	1	9.666667	Short Trip (0-1 mi)
4	1	2023-02-01 00:16:02	2023-02-01 00:22:02	1.0	0.90	1.0	N	114	148	1	..	2.5	0.0	2023-02-01	0	Wednesday	2	1	8.777778	Short Trip (0-1 mi)
5	2	2023-02-01 00:08:39	2023-02-01 00:10:50	3.0	0.53	1.0	N	263	263	1	..	2.5	0.0	2023-02-01	0	Wednesday	2	1	9.622642	Short Trip (0-1 mi)

5 rows x 27 columns

3.2.14. Analyse the trends in passenger count



3.2.15. Analyse the variation of passenger counts across zones

Zones GeoDataFrame with average passenger count per zone (head):

OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	\
0	1	0.116357	0.000782	Newark Airport	1
1	2	0.433470	0.004866	Jamaica Bay	2
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3
3	4	0.043567	0.000112	Alphabet City	4
4	5	0.092146	0.000498	Arden Heights	5

borough	geometry	\
0 EWR	POLYGON ((933100.918 192536.086, 933091.011 19...	
1 Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343...	
2 Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2...	
3 Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20...	
4 Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144...	

PULocationID_x	trip_count	PULocationID_y	avg_pickup_passenger_count	
0	1.0	5.0	1.0	2.000000
1	NaN	0.0	NaN	0.000000
2	NaN	0.0	NaN	0.000000
3	4.0	238.0	4.0	1.378151
4	NaN	0.0	NaN	0.000000

PULocationID	avg_pickup_passenger_count	
0	1	2.000000
1	4	1.378151
2	6	2.000000
3	7	1.225806
4	9	1.000000

3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

```
Surcharge Application Frequency:  
extra: 157830 trips  
mta_tax: 250517 trips  
tolls_amount: 20241 trips  
improvement_surcharge: 252089 trips  
congestion_surcharge: 235498 trips  
Airport_fee: 22308 trips
```

4. Conclusions

4.1. Final Insights and Recommendations

Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

Taxi Routing and Dispatch Optimization

- Taxi demand is highest during **morning (7–10 AM)** and **evening (4–8 PM)** hours.

- **Late-night demand** is lower but concentrated in **airports, hotels, and nightlife areas**.
- Business districts, transport hubs, and entertainment zones are the busiest locations.
- Some areas face **taxi shortages**, while others have **idle taxis**, causing imbalance.
- Certain routes are **consistently slow**, especially during rush hours.
- Passenger numbers are higher for **airport and night trips** and lower during **weekday mornings**.

Key Recommendations

- Place taxis in high-demand areas **before peak hours**.
- Move taxis from low-demand zones to high-demand zones.
- Avoid slow routes during busy periods.
- Match vehicle size to passenger demand.
- Use different dispatch strategies for peak, night, and off-peak hours.

Benefits

- Shorter wait times
- Better taxi utilization
- Less fuel waste
- Improved driver earnings
- More reliable service

Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

Strategic Cab Positioning

- Place more cabs during peak hours (morning and evening) in business areas and transport hubs.
- Focus cab availability in high-demand zones like city centers and airports.
- At night, concentrate cabs near nightlife areas, hotels, and airports.
- On weekends, shift cabs from office areas to entertainment and tourist zones.
- Move cabs dynamically from low-demand areas to high-demand areas.
- Use demand patterns to reposition cabs before shortages occur.

Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

Data-Driven Pricing Strategy

- Raise prices during **peak hours** and in **high-demand locations**.
- Apply **surge pricing carefully** to avoid customer dissatisfaction.
- Offer **lower prices during off-peak hours** to boost demand.

- Review and simplify **surcharges** to keep pricing fair and transparent.
- Adjust fares based on **trip distance, duration, and time of day**.
- Use **tipping behavior** to understand customer willingness to pay.